

Scikit-learn Pipelines

One-page cheat sheet: structure, fit/predict flow, and why we use a Pipeline

1. What is a Pipeline?

A Pipeline chains several steps (preprocessing + model) into ONE object. You call `fit()` or `predict()` once, and scikit-learn runs every step in the right order for you.

```
pipeline = Pipeline([
    ('preprocessor', preprocessor), # step 1: prepare the data
    ('model', RandomForestRegressor()) # step 2: learn / predict
])
```

2. Structure of the pipeline

```
pipeline
|-- preprocessor
|   |-- num -> numerical_pipeline
|   |       (imputer, scaler)
|   |-- cat -> categorical_pipeline
|           |-- imputer
|           |-- encoder
|-- model
    |-- RandomForestRegressor
```

num: numeric columns get imputed (fill missing) then scaled.

cat: categorical columns get imputed then one-hot encoded.

model: receives the fully processed data and learns.

3. Why not do it manually?

Without Pipeline (you manage it)

```
imputer = SimpleImputer(strategy='mean')
scaler = StandardScaler()
```

```
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

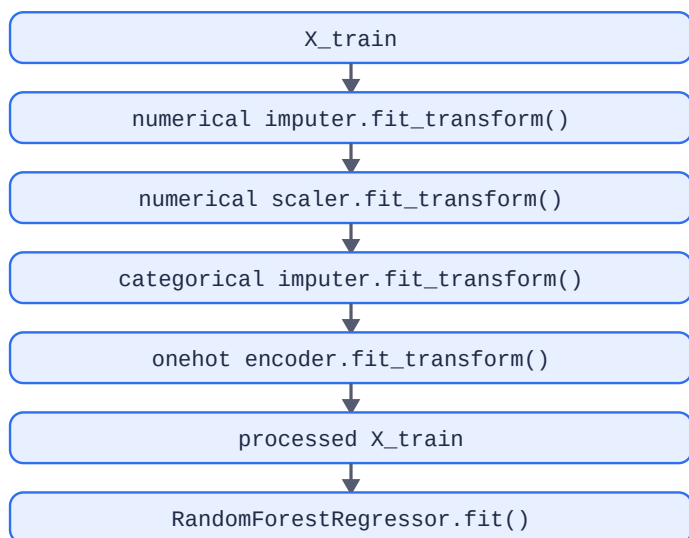
With Pipeline (it handles it)

```
pipeline.fit(X_train, y_train)
preds = pipeline.predict(X_test)
```

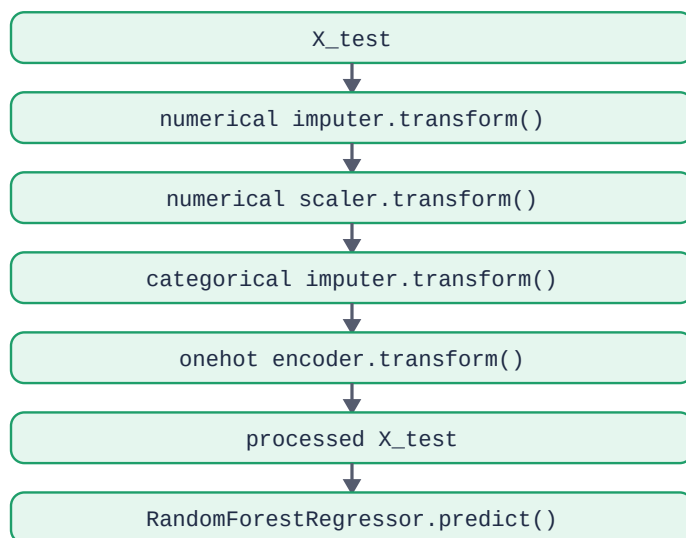
Fewer lines, no mistakes in the order of steps, and the exact same processing is reused on any new data.

4. What happens inside?

pipeline.fit(X_train, y_train)



pipeline.predict(X_test)



Golden rule

`fit` / `fit_transform` learns from TRAIN data only. On test data we only transform / predict, using what was learned from train. The Pipeline enforces this automatically, which prevents data leakage.

Evaluation Metrics

Each model type has its own metrics, and the Pipeline does not change them

1. Key idea

The metric depends on the TASK (what the final model predicts), not on the Pipeline. A Pipeline is only the packaging of the steps. Preprocessing steps (imputer, scaler, encoder) are not evaluated; only the final model's predictions are compared with the true values.

2. Metrics by model type

Task / example model	Common metrics	What it tells you
Regression (RandomForestRegressor, LinearRegression)	MAE, MSE, RMSE, R2	Size of the error between predicted and real numbers. R2 = how much variance is explained (1 is best).
Classification (RandomForestClassifier, LogisticRegression)	Accuracy, Precision, Recall, F1, ROC-AUC, Confusion matrix	How often the predicted class is right, and what kind of mistakes are made.
Clustering (KMeans, DBSCAN)	Silhouette score, Inertia	How tight and well separated the clusters are (no true labels needed).

3. Metric formulas (simple)

MAE	mean of $ \text{actual} - \text{predicted} $	Average error, same unit as the target
MSE	mean of $(\text{actual} - \text{predicted})^2$	Punishes big errors more
RMSE	square root of MSE	Like MAE but sensitive to large errors
Precision	$\text{TP} / (\text{TP} + \text{FP})$	Of predicted positives, how many were right
Recall	$\text{TP} / (\text{TP} + \text{FN})$	Of real positives, how many we found
F1	$2 * \text{P} * \text{R} / (\text{P} + \text{R})$	Balance between precision and recall

4. Evaluating a pipeline in code

```
pipeline.fit(X_train, y_train)
preds = pipeline.predict(X_test)

# Regression (RandomForestRegressor)
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae = mean_absolute_error(y_test, preds)
rmse = mean_squared_error(y_test, preds) ** 0.5
r2 = r2_score(y_test, preds)

# Classification (if the model were a classifier)
from sklearn.metrics import accuracy_score, f1_score
acc = accuracy_score(y_test, preds)
```

5. Whole workflow in one line each

